

Due date : 19 May 2026

Bright Coffee Shop Sales SQL Exercise: Wildcards & Window Functions Databricks Hands-On Worksheet

Testing and checking table insights

```
select * from `workspace`.`default`.`bright_coffee_shop_sales`  
    May` limit 100;
```

---- Question 1—

```
SELECT transaction_id, product_detail, unit_price  
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`  
WHERE product_detail LIKE 'dark%'  
LIMIT 10;
```

How many distinct product_detail values start with 'Dark'? (run COUNT DISTINCT to find out)

---No rows returned----

---Question 2-----

```
SELECT transaction_id, product_detail, product_category, unit_price  
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`  
WHERE product_detail LIKE '%Lg'  
ORDER BY unit_price DESC;
```

-----Question 3-----

```
SELECT DISTINCT
    product_type,
    product_detail
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
WHERE product_detail LIKE '%Chai%'
ORDER BY product_detail;
```

----Question 4-----

```
SELECT DISTINCT product_category, product_type, product_detail
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
WHERE product_category NOT LIKE '%Tea%'
ORDER BY product_category, product_type;
```

----Question 5-----

```
SELECT DISTINCT product_detail
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
WHERE product_detail LIKE '%__'
ORDER BY product_detail;
```

What is the difference between % and _ in a LIKE pattern? Give one example of each using this dataset.

% percentage matches any number WHERE product_name LIKE 'chai%'
_underscore matches one character WHERE product_name LIKE 'chai_'

----Question 6-----

```
SELECT DISTINCT product_detail
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
WHERE product_detail RLIKE '[0-9]'
ORDER BY product_detail;
```

----Question7----

```
SELECT transaction_id, transaction_date, store_location,
       product_detail, unit_price
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
WHERE product_category = 'Coffee'
      AND (product_detail LIKE '%Brazilian%' OR product_detail LIKE '%Ethiopian%')
ORDER BY transaction_date;
```

-----Question 8-----

Explain in your own words what the following pattern matches: `product_detail LIKE '%Scone'`
Then write a different LIKE pattern that would match product names that contain the word 'Blend' anywhere in the middle of the name.

The pattern `product_detail LIKE '%Scone'` matches with any product wording that ends with the word "scone"

% means : wildcard 0 or multiple character before the word, this allows any additional character before and after

Scone is the last part of the string because there is no wild card

Matches :scone,Blueberry scone,plain scone

No Matches: chocolate scone recipe,scone,scone recipe

----Question 9-----

```
SELECT
transaction_id,
store_location,
transaction_qty,
unit_price,
SUM(transaction_qty * CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2))) OVER(PARTITION BY
store_location) AS store_total_revenue
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
LIMIT 20;
```

The PARTITION BY store_location clause groups the data by each individual store.

The Aggregate Behaviour: The SUM() function calculates one grand total for that entire store group.

The Window Function Behaviour: Instead of collapsing the rows (like a traditional GROUP BY), the window function broadcasts that exact same total revenue figure across every single individual transaction row belonging to that specific store.

-----Question 10-----

```
SELECT
transaction_id,
store_location,
ROUND(transaction_qty * CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2)), 2) AS revenue,
ROUND( (transaction_qty * CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2))) * 100.0 /
SUM(transaction_qty * CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2))) OVER(PARTITION BY
store_location) , 2) AS pct_of_store_revenue
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
LIMIT 20;
```

-----Question 11---

```
SELECT
transaction_id,
store_location,
transaction_date,
transaction_time,
ROUND(transaction_qty * CAST(REPLACE(unit_price, ',', '.') AS DECIMAL(10,2)), 2) AS revenue,
ROUND( SUM(transaction_qty * CAST(REPLACE(unit_price, ',', '.') AS DECIMAL(10,2))) OVER(
PARTITION BY store_location
ORDER BY transaction_date, transaction_time
), 2) AS running_total
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
LIMIT 30;
```

What is the key difference between the SUM() in Q9 (no ORDER BY) and the SUM() in Q11 (with ORDER BY)? What does adding ORDER BY change about how the window is calculated?

The key difference : Q9 and Q11

The Key Difference: Q9 vs. Q11

The key difference is that Q9 calculates a static grand total for the entire store, while Q11 calculates a dynamic cumulative total that builds up line by line.

What ORDER BY Changes About the Window Calculation

Adding ORDER BY changes the default window frame (the rows the function looks at):

- Without ORDER BY (Q9): The window includes every single row in the partition at once. The database calculates the total for the whole store and applies that identical number to every row.
- With ORDER BY (Q11): The window frame changes to a running frame (RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW). The database sorts the transactions chronologically and calculates the sum only from the start of the partition up to the current row. This causes the total to increase sequentially with each new transaction.

---Question 12---

```
SELECT
transaction_id,
store_location,
ROUND(transaction_qty * CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2)), 2) AS revenue,
ROW_NUMBER() OVER( PARTITION BY store_location ORDER BY transaction_qty *
CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2)) DESC ) AS row_num
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
ORDER BY store_location, row_num
LIMIT 30;
```

----Question 13---

```
SELECT
product_category,
product_detail,
unit_price,
ROW_NUMBER() OVER( PARTITION BY product_category ORDER BY unit_price DESC ) AS rn,
RANK() OVER( PARTITION BY product_category ORDER BY unit_price DESC ) AS rnk,
DENSE_RANK() OVER( PARTITION BY product_category ORDER BY unit_price DESC ) AS d_rnk
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
ORDER BY product_category, unit_price DESC
LIMIT 40;
```

[WRITTEN] Find a tie in your results (two rows with the same unit_price in the same category). Write down the values of rn, rnk, and d_rnk for those rows. Explain in one sentence why RANK and DENSE_RANK give the same result for tied rows but differ for the row after the tie.

Example Tie Values

If two identical espresso drinks both cost R3.50 within the "Coffee" category, your results will look like this:

product_category	product_detail	unit_price	rn	rnk	d_rnk
Coffee	Latte	3.50	1	1	1
Coffee	Cappuccino	3.50	2	1	1
Coffee	Cortado	3.00	3	3	2

Explanation of the Row After the Tie

RANK and DENSE_RANK assign the same number to tied rows because they both evaluate identical values equally, but for the row after the tie, RANK skips ranks based on the number of tied items ($1 + 1 \text{ tie} = 3$) to show true relative position, whereas DENSE_RANK always continues with the next immediate sequential integer ($1 \rightarrow 2$) without gaps.

----Question 14----

```

SELECT
transaction_id,
store_location,
ROUND(transaction_qty * CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2)), 2) AS revenue,
NTILE(4) OVER(PARTITION BY store_location ORDER BY transaction_qty *
CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2)) ASC) AS quartile
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
ORDER BY store_location, revenue ASC
LIMIT 30;

```

----Question 15-----

```
SELECT
transaction_id,
store_location,
transaction_date,
ROUND(transaction_qty * CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2)), 2) AS revenue,
ROUND( LAG(transaction_qty * CAST(REPLACE(unit_price, ',', '')) AS DECIMAL(10,2)), 1, 0) OVER(
PARTITION BY store_location ORDER BY transaction_date, transaction_time
), 2) AS prev_revenue
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
LIMIT 20;
```

----Question 16-----

```
SELECT
transaction_id,
store_location,
transaction_date,
ROUND(transaction_qty * unit_price, 2) AS revenue,
ROUND( LEAD(transaction_qty * unit_price, 1) OVER
PARTITION BY Store_location ORDER BY transaction_date,transacion_time,
), 2) AS next_revenue
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
LIMIT 20;
```


---Question 17----

```
SELECT DISTINCT
product_category,
product_detail,
unit_price,
FIRST_VALUE( product_detail ) OVER(
PARTITION BY product_category ORDER BY unit_price ASC
) AS cheapest_in_category
FROM `workspace`.`default`.`bright_coffee_shop_salesMay`
ORDER BY product_category, unit_price
LIMIT 30;
```

---Question 18----

```
SELECT
transaction_id,
store_location,
product_detail,
ROUND(transaction_qty * unit_price, 2) AS revenue,
RANK() OVER(PARTITION BY store_location ORDER BY transaction_qty * unit_price DESC) AS
store_rank,
ROUND( AVG(transaction_qty * unit_price) OVER(PARTITION BY store_location) , 2) AS
store_avg_revenue
FROM coffee_sales
WHERE product_detail LIKE '%Lg'
      OR product_detail LIKE '%Rg'
ORDER BY store_location, store_rank
LIMIT 40;
```

[WRITTEN] Look at your results. Does every store_avg_revenue change between rows? Explain why or why not. Then explain in one sentence what store_rank = 1 means for each store.

Does store_avg_revenue change between rows?

No, the store_avg_revenue does not change between rows for the same store.

Why or why not?

The PARTITION BY store_location clause locks the calculation window to each specific store group, meaning the AVG() function calculates a single, fixed baseline average for that entire store. Because there is no ORDER BY clause inside the AVG() window, it defaults to evaluating all rows in the partition at once and outputs that exact same static average across every line belonging to that store.

Meaning of store_rank = 1

A store_rank = 1 means that the specific row represents the transaction that generated the single highest revenue amount for that individual store.